Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопсности
Высшая школа программной инженерии

ЛАБОРАТОРНАЯ РАБОТА №7

по дисциплине «Теория автоматов и формальных языков»

Выполнил
студент гр. 5130904/30108 Ребдев П.А.

Проверил Тышкевич А.И.

Санкт-Петербург
2026

1. Формулировка задания

1. Задать контекстно-свободную грамматику для языка арифметических выражений (калькулятор), выполняющего операции сложения, вычитания, умножения, деления и унарный минус.
2. Определить атрибутную семантику, позволяющую вычислять значение выражения.
3. Проверить грамматику на пригодность для детерминированного нисходящего разбора (LL(1)). При необходимости выполнить преобразования.
4. Привести пример вычисления по дереву вывода для конкретного слова.
5. Разработать транслятор методом рекурсивного спуска, реализующий вычисления, и показать на диаграммах места вставки семантических действий.
6. Продемонстрировать протокол вычислений при трансляции выбранного слова.

2. Исходная грамматика и её преобразование

Для описания арифметических выражений с учётом приоритета операций и левоассоциативности естественно использовать грамматику с левой рекурсией:

Expr → Expr ('+'|'-') Term | Term
Term → Term ('*'|'/') Factor | Factor
Factor → number | '(' Expr ')' | '-' Factor

Данная грамматика не является LL(1) из-за левой рекурсии. Для нисходящего разбора необходимо устранить левую рекурсию и, при необходимости, выполнить левую факторизацию. После преобразования получаем эквивалентную LL(1)-грамматику:

№	Продукция	Пояснение
1	E → T E'	Начало выражения
2	E' → + T E'	Сложение
3	E' → - T E'	Вычитание
4	E' → ε	Конец выражения
5	T → F T'	Начало терма
6	T' → * F T'	Умножение
7	T' → / F T'	Деление
8	T' → ε	Конец терма
9	F → number	Число
10	F → (E)	Скобочное выражение
11	F → - F	Унарный минус

Терминалы: number, +, -, *, /, (,), $ (маркер конца).
Нетерминалы: E, E', T, T', F.
Начальный символ: E.

3. Проверка LL(1)-свойства

Для каждой продукции вычислены множества FIRST и FOLLOW, а затем LOOKAHEAD.

3.1. FIRST

Нетерминал	FIRST
E	{ number, (, - }
E'	{ +, -, ε }
T	{ number, (, - }
T'	{ *, /, ε }
F	{ number, (, - }

3.2. FOLLOW

Нетерминал	FOLLOW
E	{ $,) }
E'	{ $,) }
T	{ +, -, $,) }
T'	{ +, -, $,) }
F	{ *, /, +, -, $,) }

3.3. LOOKAHEAD для продукций с несколькими альтернативами

- **E' → + T E' : LOOKAHEAD = {+}**

E' → - T E' : LOOKAHEAD = {-}

E' → ε : LOOKAHEAD = FOLLOW(E') = {$,)}

Пересечений нет.

- **T' → * F T' : LOOKAHEAD = {*}**

T' → / F T' : LOOKAHEAD = {/}

T' → ε : LOOKAHEAD = FOLLOW(T') = {+, -, $,)}

Пересечений нет.

- **F → number : LOOKAHEAD = {number}**

F → (E) : LOOKAHEAD = {(}

F → - F : LOOKAHEAD = {-}

Пересечений нет.

Все условия выполнены → грамматика является LL(1) и пригодна для рекурсивного спуска.

4. Атрибутная семантика

Каждому символу грамматики сопоставлены семантические атрибуты.

Для нетерминалов E, T, F определён синтезируемый атрибут val – числовое значение.

Для нетерминалов E' и T' определены наследуемый атрибут inh (промежуточное значение, переданное слева) и синтезируемый атрибут syn (итоговое значение).

В реализации рекурсивного спуска наследуемые атрибуты передаются как параметры функций, синтезируемые возвращаются как результат.

Семантические правила:

$E \rightarrow T\ E'$: $E'.inh = T.val$; $E.val = E'.syn$

$E' \rightarrow +\ T\ E'_1$: $E'_1.inh = E'.inh + T.val$; $E'.syn = E'_1.syn$

$E' \rightarrow -\ T\ E'_1$: $E'_1.inh = E'.inh - T.val$; $E'.syn = E'_1.syn$

$E' \rightarrow \varepsilon$: $E'.syn = E'.inh$

$T \rightarrow F\ T'$: $T'.inh = F.val$; $T.val = T'.syn$

$T' \rightarrow *\ F\ T'_1$: $T'_1.inh = T'.inh * F.val$; $T'.syn = T'_1.syn$

$T' \rightarrow /\ F\ T'_1$: $T'_1.inh = T'.inh / F.val$ (целочисленное деление); $T'.syn = T'_1.syn$

$T' \rightarrow \varepsilon$: $T'.syn = T'.inh$

$F \rightarrow number$: $F.val$ = значение числа (из лексера)

$F \rightarrow (\ E\ )$: $F.val = E.val$

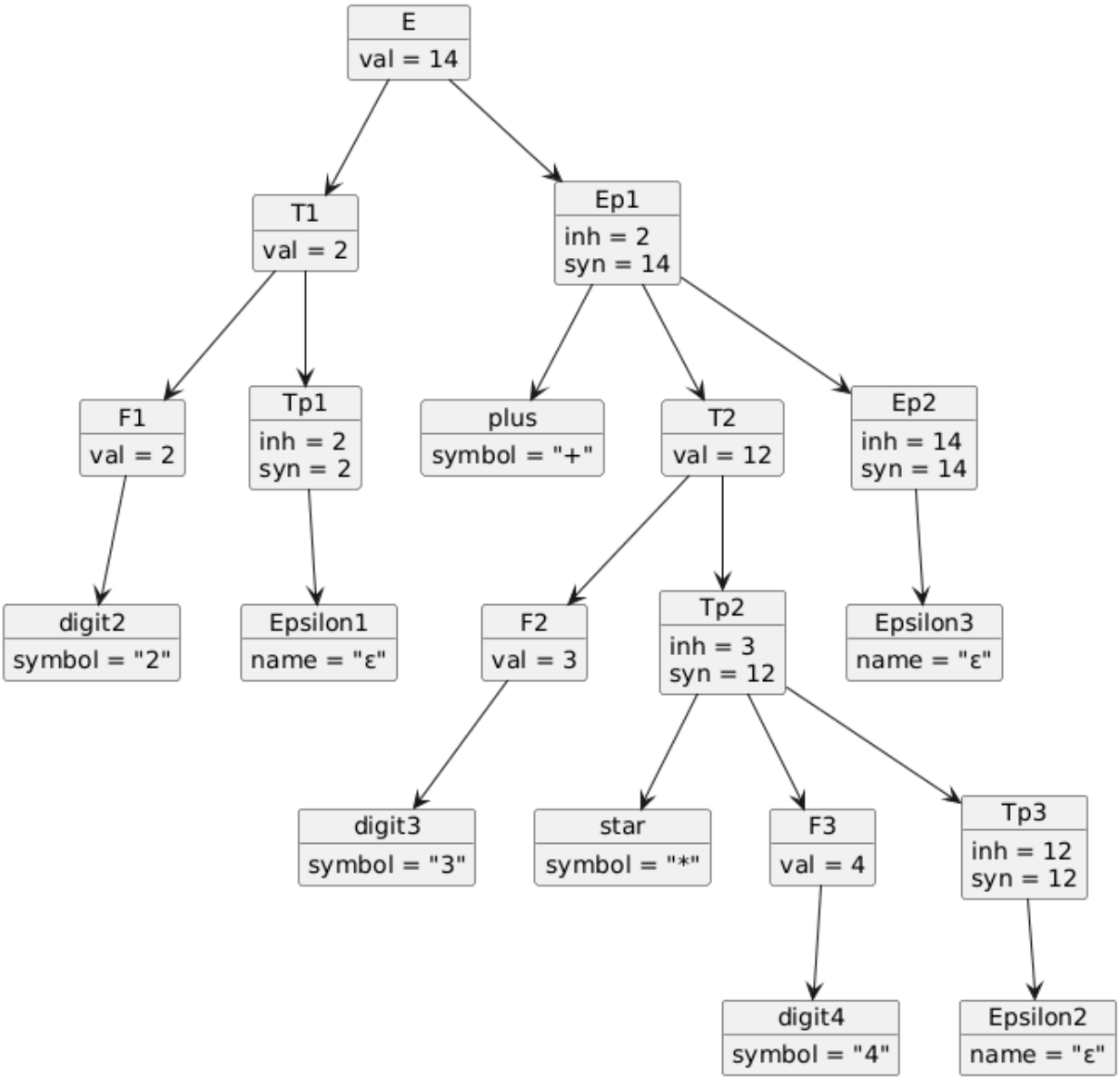
$F \rightarrow -\ F_1$: $F.val = -\ (F_1.val)$

Граф зависимостей ацикличен, вычисления выполняются при обходе в порядке рекурсивного спуска.

5. Пример вычисления по дереву вывода

Рассмотрим слово (входную цепочку) 2 + 3 * 4 $. Лексемы: number(2), '+', number(3), '*', number(4), $.

Дерево вывода с атрибутами представлено на диаграмме ниже. Каждый объект соответствует узлу дерева, указаны значения атрибутов. Терминалы изображены в виде объектов с символом, ε-переходы – как объекты Epsilon.



Пошаговое вычисление (соответствует диаграмме):

1. F1 → number(2) : F1.val = 2
2. T1 → F1 Tp1 : Tp1.inh = 2, затем Tp1 → ε → Tp1.syn = 2, T1.val = 2
3. E → T1 Ep1 : Ep1.inh = 2
4. Видим '+', выбираем Ep1 → + T2 Ep2:
   - T2 → F2 Tp2 : F2 → number(3) → F2.val = 3, Tp2.inh = 3

- Далее Tp2 → * F3 Tp3 : F3 → number(4) → F3.val = 4, Tp3.inh = 3 * 4 = 12, затем Tp3 → ε → Tp3.syn = 12, следовательно Tp2.syn = 12, T2.val = 12
- Теперь Ep2.inh = Ep1.inh + T2.val = 2 + 12 = 14, затем Ep2 → ε → Ep2.syn = 14, значит Ep1.syn = 14

5. E.val = Ep1.syn = 14

Результат: 14, что соответствует математическому значению 2 + 3 * 4.

6. Реализация транслятора методом рекурсивного спуска

6.1. Диаграммы рекурсивного спуска с указанием мест семантических действий

Диаграмма для E

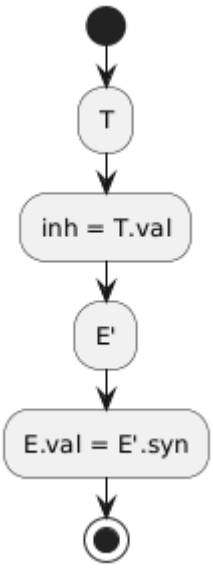


Диаграмма для E'

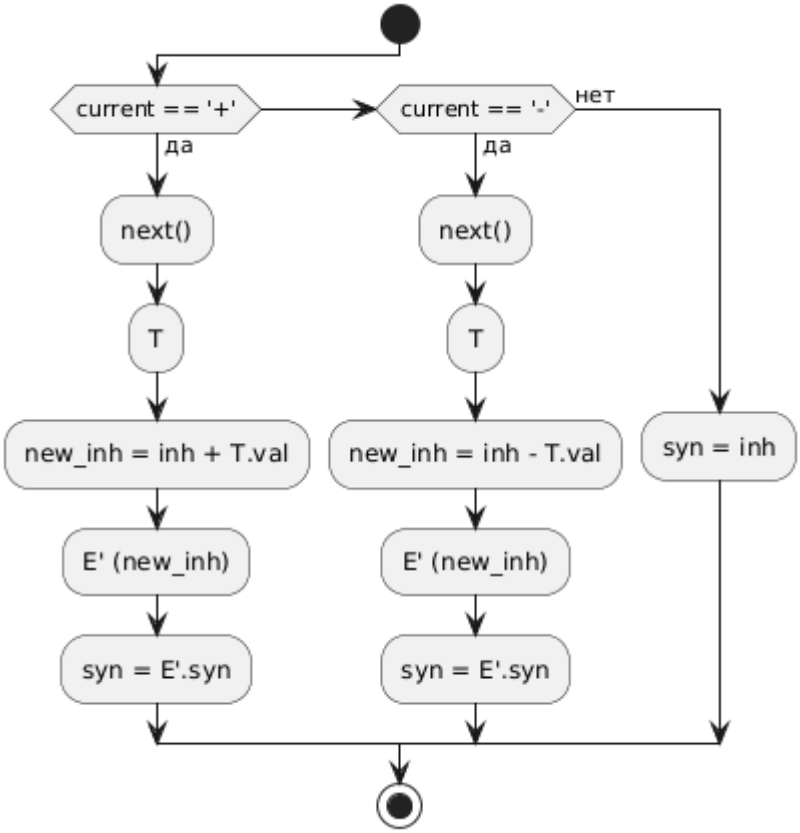


Диаграмма для T

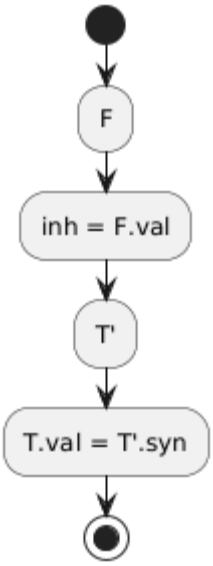


Диаграмма для T'

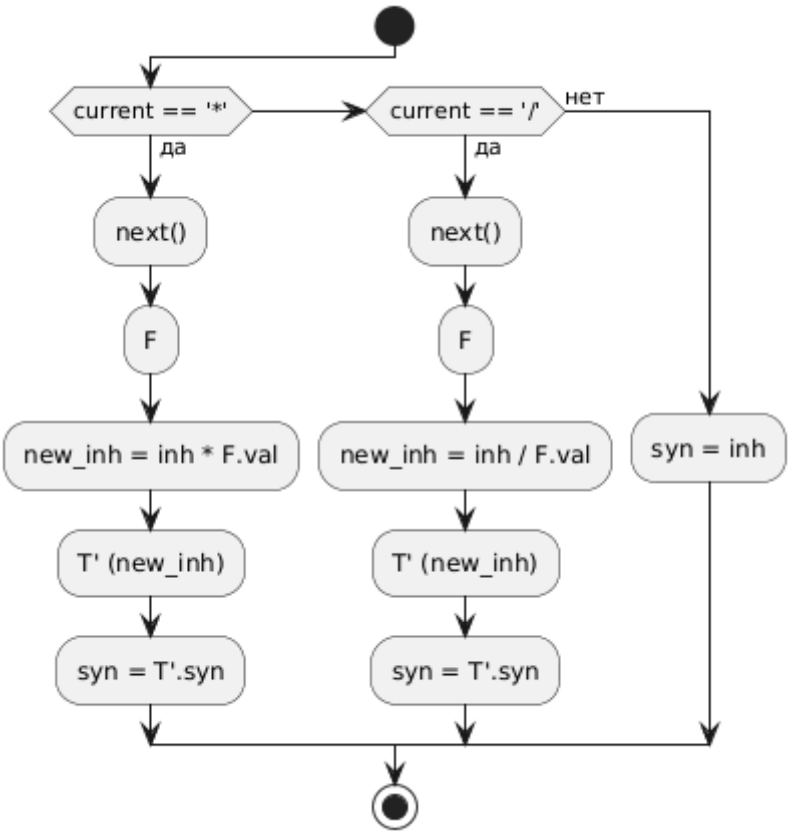
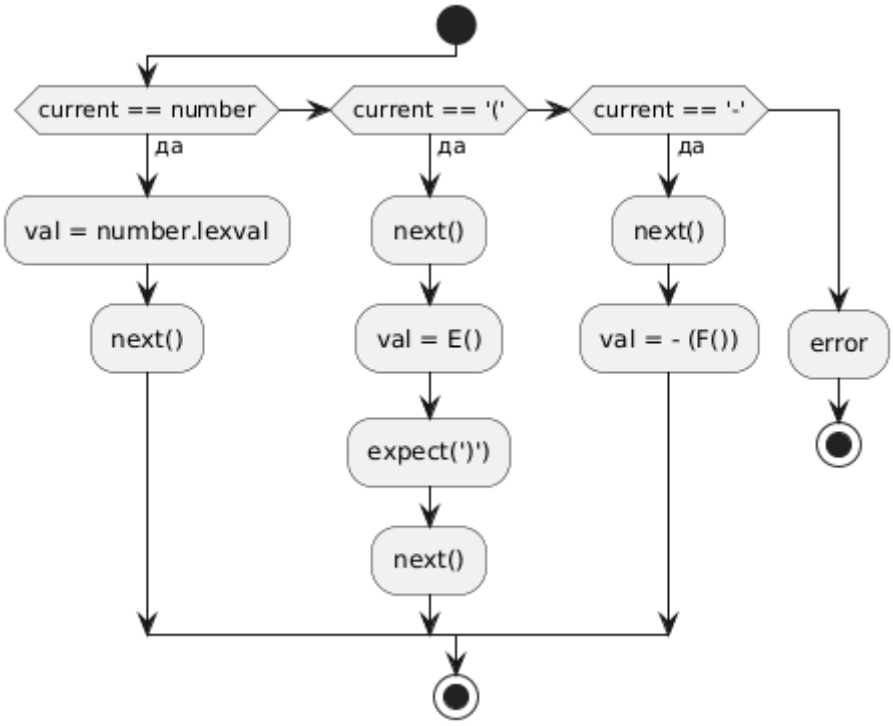


Диаграмма для F



6.2. Программная реализация на C++

Программа на C++ читает выражение, заканчивающееся символом $, вычисляет его значение и выводит результат. Лексический анализ выполняется вручную. Для отладки добавлена трассировка с отступами.

```
#include <iostream>
#include <string>
#include <cctype>
#include <cstdlib>

std::string input;
size_t pos;
int token;
int numval;
int depth;

void next()
{
  while (pos < input.size() && std::isspace(input[pos]))
  {
    ++pos;
  }
  if (pos >= input.size())
  {
    token = '$';
    return;
  }
  if (std::isdigit(input[pos]))
  {
    numval = 0;
    while (pos < input.size() && std::isdigit(input[pos]))
    {
      numval = numval * 10 + (input[pos] - '0');
      ++pos;
    }
    token = 'n';
  }
```

```
  else
  {
    token = input[pos];
    ++pos;
  }
}

void printIndent()
{
  for (int i = 0; i < depth; ++i)
  {
    std::cout << "  ";
  }
}

int E();
int E_prime(int inh);
int T();
int T_prime(int inh);
int F();

int E()
{
  printIndent();
  std::cout << "E: enter, token=" << char(token) << "\n";
  ++depth;
  int tval = T();
  int res = E_prime(tval);
  --depth;
  printIndent();
  std::cout << "E: return " << res << "\n";
  return res;
}

int E_prime(int inh)
{
  printIndent();
  std::cout << "E': enter, inh=" << inh << ", token=" << char(token) << "\n";
  ++depth;
  int res;
  if (token == '+')
  {
    next();
    int t = T();
    res = E_prime(inh + t);
  }
  else if (token == '-')
  {
    next();
    int t = T();
    res = E_prime(inh - t);
  }
  else
  {
    res = inh;
  }
  --depth;
  printIndent();
  std::cout << "E': return " << res << "\n";
  return res;
}
```

```
int T()
{
  printIndent();
  std::cout << "T: enter, token=" << char(token) << "\n";
  ++depth;
  int f = F();
  int res = T_prime(f);
  --depth;
  printIndent();
  std::cout << "T: return " << res << "\n";
  return res;
}

int T_prime(int inh)
{
  printIndent();
  std::cout << "T': enter, inh=" << inh << ", token=" << char(token) << "\n";
  ++depth;
  int res;
  if (token == '*')
  {
    next();
    int f = F();
    res = T_prime(inh * f);
  }
  else if (token == '/')
  {
    next();
    int f = F();
    if (f == 0)
    {
      std::cerr << "Error: division by zero\n";
      std::exit(1);
    }
    res = T_prime(inh / f);
  }
  else
  {
    res = inh;
  }
  --depth;
  printIndent();
  std::cout << "T': return " << res << "\n";
  return res;
}

int F()
{
  printIndent();
  std::cout << "F: enter, token=" << char(token) << "\n";
  ++depth;
  int val;
  if (token == 'n')
  {
    val = numval;
    next();
  }
  else if (token == '(')
  {
    next();
    val = E();
    if (token != ')')
```

```
    {
      std::cerr << "Error: expected ')'\n";
      std::exit(1);
    }
    next();
  }
  else if (token == '-')
  {
    next();
    val = -F();
  }
  else
  {
    std::cerr << "Error: unexpected token " << char(token) << "\n";
    std::exit(1);
  }
  --depth;
  printIndent();
  std::cout << "F: return " << val << "\n";
  return val;
}

int main()
{
  std::cout << "Enter expression (end with $): ";
  std::getline(std::cin, input);
  pos = 0;
  next();
  int result = E();
  if (token != '$')
  {
    std::cerr << "Error: unexpected characters after expression\n";
    return 1;
  }
  std::cout << "Result = " << result << "\n";
  return 0;
}
```

6.3. Соответствие диаграммам

Каждая функция E, E', T, T', F реализует соответствующую диаграмму. В точках выбора альтернатив проверяется текущий токен (LOOKAHEAD), после чего выполняются семантические действия: для E' и T' вычисляется новое наследуемое значение и делается рекурсивный вызов, для F возвращается значение числа или результат подвыражения. Отладочная печать с отступами показывает глубину рекурсии и значения на входе и выходе, что полностью соответствует протоколу обхода дерева.

7. Протокол вычислений

Пример 1: 2+3*4$

```
Enter expression (end with $): E: enter, token=n
  T: enter, token=n
    F: enter, token=n
    F: return 2
    T': enter, inh=2, token=+
    T': return 2
  T: return 2
  E': enter, inh=2, token=+
    T: enter, token=n
      F: enter, token=n
      F: return 3
      T': enter, inh=3, token=*
        F: enter, token=n
        F: return 4
        T': enter, inh=12, token=$
        T': return 12
      T': return 12
    T: return 12
    E': enter, inh=14, token=$
    E': return 14
  E': return 14
E: return 14
Result = 14
```

Пример 2 (с унарным минусом): -2+3*4$

```
Enter expression (end with $): E: enter, token=-
  T: enter, token=-
    F: enter, token=-
      F: enter, token=n
      F: return 2
    F: return -2
    T': enter, inh=-2, token=+
    T': return -2
  T: return -2
  E': enter, inh=-2, token=+
    T: enter, token=n
      F: enter, token=n
      F: return 3
      T': enter, inh=3, token=*
        F: enter, token=n
        F: return 4
        T': enter, inh=12, token=$
        T': return 12
      T': return 12
    T: return 12
    E': enter, inh=10, token=$
    E': return 10
  E': return 10
E: return 10
Result = 10
```

Протокол полностью соответствует обходу дерева вывода и демонстрирует корректную обработку всех операций, включая унарный минус и деление (с проверкой на ноль).

8. Выводы

1. Разработана LL(1)-грамматика для арифметических выражений, пригодная для рекурсивного спуска.
2. Задана атрибутная семантика с наследуемыми и синтезируемыми атрибутами, позволяющая вычислять значение выражения.
3. На примере 2+3*4 построено дерево вывода в UML и продемонстрировано вычисление.
4. Реализован транслятор на C++, в точности следующий диаграммам рекурсивного спуска и удовлетворяющий всем требованиям к стилю кодирования.
5. Получен протокол вычислений, подтверждающий корректность работы.

Транслятор может быть легко расширен для поддержки дополнительных операций и функций.